The tools and techniques of R programming
Lesson-2
R as a Programming Environment

By
Prof.Dr.A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

August 22, 2023

OBJECTIVES OF THIS LESSON

This lesson has been developed with a view to achieve the following objectives:

- **Basic Concepts of Programming**
- **Concepts of different kinds of system software required for the execution process of a program**
- **Concepts of Structured Programs**
- **Standard program logic structures**
- **Concepts of unstructured Programs**
- **Illustrative examples to demonstrate the 'sequence logic structure'**

Basics of Programming

R provides an interpreted programming environment. To understand this, let us have a brief discussion on programming basics.
We know that **a computer program is a sequential set of instructions in a programming language which is usually executed in a batch mode with a view to generate some desired output by a computer on the basis of some appropriate input**.The instructions of a program are written one after another in the order of execution following some predefined algorithm by using a text editor and then saved in a file;this unit of a computer program is called a **source program**.The source program is then translated in its equivalent machine language form by using either an **assembler**[For instructions in an Assembly Language] or a **compiler**[For instructions in a HLL], as the case may be,to generate the translated version of the source program which is called an **object module** or **object program**.The object program is then linked with the standard library routines leading to the formation of an **executable module** by using a system software called a **Linker**. The executable module can then be loaded in the computer's main memory any number of times for execution of the steps specified in the program, called program logic, by using a system software called **loader**. Inputs are then supplied, if required and output is obtained in a successful run.

Basics of Programming–Contd.

The translator that we use for assembling or compilation understands only the syntax of the language; so, it shows a message **"No errors or warnings"** when it finds no syntactic error in the instructions of a program. However, the program may contain **semantic errors**, also called **logical errors**, which are understood by the computer and on facing such an error, the machine terminates the program showing some message. Such a program is to be checked again starting from the source code and the steps described above are to be followed again to create the executable code set. This process is followed in assembly language programming and in some high level language programming like **COBOL**, **C** etc.

There is an alternative translator called **interpreter** that translates an instruction in a programming language by checking its syntax as well as semantics and then executes the instruction immediately, if the instruction is found to be correct in both respects to generate the output meant by the instruction. Thus an interpreter handles a program through translation and execution of one instruction at a time until it reaches the end of the set of codes submitted.

Basics of Programming–Contd.

R,being **a scripting language** provides us with an environment where each entered instruction is taken care of by the in-built command interpreter for immediate translation followed by its execution.Such an environment is being referred to as **interpreted programming environment**. So, the environment readily provided by R is an interactive one. This enables us to follow the incremental development process in which we can write only one instruction at a time,test it and go for writing the next statement if the preceding one is found to be correct.However,**in R, we can also perform batch processing of instructions** where we can write all the requisite instructions to implement some algorithm using an in-built editor in the system which are then submitted to the R interpreter either partly or wholly for the desired interpretation of the statements one by one till the end of the batch if no errors are encountered in any step.A complete set of instructions implementing an algorithm is termed as an **R Script**. By the definition, we can have a single instruction also to form an R script if it can implement an algorithm.**Hence, R can be used both as an interactive as well as a batch processing programming language**.

Basics of Programming–Contd.

In an interactive mode, we see the **non-procedural feature of R** in which we need not state **'how to do something'** but **'what to do'**.Such computer languages are known as **fourth generation languages(4 GLs)**. On the contrary, in a batch mode of programming, we specify the tasks to be performed sequentially, i.e., one after another to achieve a particular objective, i.e., we state,**'how to do something'**. This is the basic feature of a procedural language which is also known as **third generation language(3 GL)**.R was created as a computer language with both the features of **3 GL** as well as **4 GL**. We will explore first the **3 GL** features of R in a few lessons now. Later, we will also study the 4 GL features in a greater detail in the following lessons.

The 3 GL features of R can also be utilised to develop **structured programs or systems** which is very essential and desirable form of programming technique for all types of data processing including statistical analysis to data analytics and data mining apart from normal data processing.

A program is said to be a **structured program** if it is easily, readable, easily understandable and easily modifiable.

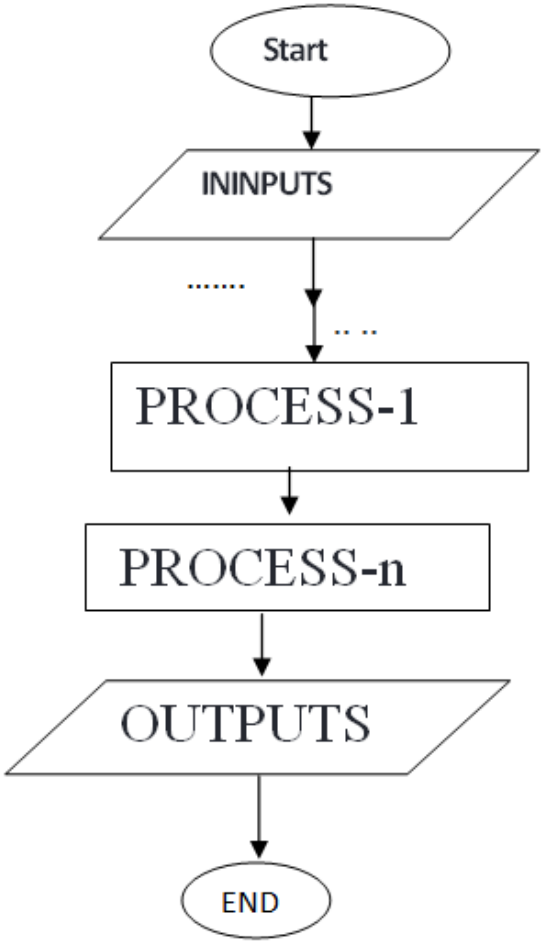
Structured programming environment

Structured programs can be developed by breaking down the activities into smaller units of tasks for performing a specific function.Each such unit is called a **module**.Each of such modules should only contain statements or instructions formed with only one or more of four standard logic structures, namely, **Sequence, Selection, Iteration and Case**.This is why structured programs are also called **moduler programs** and the **programming technique is known as moduler programming.** This is a **Top-Down approach** to program development that can materialise the characteristics of a **Structured programs**. The programs which lack the features of a structured program are known as **unstructured** of **Spaghetti-bowl programs**. Programs, particularly consisting of instructions like **"Go To"** for unconditional transfer of controls, are known as **unstructured** or **Spaghetti-bowl programs**; because such instructions make the programs a weird set of instructions that cannot be debugged easily and lose the goodness of readability and understandability.

We shall study R by defining and implementing the logic structures of structured programming one by one in the subsequent lessons.

The Sequence logic structure

This logic structure includes simple **input-process-output** statements. There may or may not be any input statement; but there should be one or more processing or output statements. The logic structure can be depicted pictorially as shown below:



Implementation of the sequence logic structure in R

To implement the sequence logic structure in R in the simplest way, we need to know how to accept input from the console , how to process the stored data and how to show the output.

The simplest input statement to accept input from the console of the user is to use a system-defined function shown below:

Identifier assignment –operator redline (prompt=" Any text you would like to display")

For example:**X=readline(prompt="Enter your Name Here=>")**

On display of the prompt, anything the user types followed by the pressing of the **ENTER** key, will be stored in the identifier X as a set of characters.

If the data comprises digits, it can be converted into a numeric data as follows:**Y=as.numeric(X)**

Now, we can utilize the value of Y to perform arithmetic computations.

An alternative to the above statement is the use of another system-defined function **scan( )** to accept one or more inputs at a time in a numeric form directly. To show any prompt with this statement, we may make use of the **cat()** function to include the desired text as illustrated below:

Implementation of the sequence logic structure in R–Contd.

cat("What is your age?")
x=scan(n=1)
The syntax of the **scan() function** is very complex and we shall discuss it at length at an appropriate stage.We show here the simplest form of the function. Now, we can make use of the value stored in **x** in a statement like: **x=x+1** .
Here **n=1** implies that we shall accept only one input. For multiple inputs, the value of n is to be specified accordingly. After the entry of each input value, the **ENTER** key is to be pressed. All the input values are stored **in a subscripted list** with the name of the identifier and then the values can be accessed as **x[1], [2],. . . etc.**,where **x** is the name of the identifier defined by the user;it is also called an **R object**.The detailed concept of the list data structure will be discussed at a later lesson.
The Process statement is nothing but an assignment statement. The format of which is:
Identifier assignment-operator variable |expression|_formula

Implementation of the sequence logic structure in R–Contd.

For example,

perimeter=2*pi*radius

Finally, the output statement to be written may be one of the following: **print(identifier)** Or, **cat(identifier)** However, if we want to show both the value of an identifier as well as some text to describe the value, the output statement will have the following form:

print(paste(identifier,"text", identifier, "text",identifier...))

Or, **cat("text",identifier)**

Here, some identifier or text may be omitted, if not required. The **ellipsis (...)** implies that the format may be used any number of times depending upon the requirement of the user. Finally, we come to the point of writing a complete program. **There is no predefined format of an R program as in c or COBOL language.** Instructions of a program are to be written just in the order in which we want to execute them. To illustrate the idea, we show below the codes of an R program to determine the area and perimeter of a circular plot.

Implementation of the sequence logic structure in R–Contd.

We can write requisite instructions one by one at the R prompt and execute it instantly to obtain the result of execution interactively;alternatively, all the statements can be written in an editor by selecting **new script** option under **new file** option under the **File** menu option of the base R or the R studio and the statements can be executed by **"Run all"** option under **Edit** menu option. There is still a third alternative in which we save the set of statements in a file with a desired filename and then the entire set of instructions is submitted for execution in the batch mode by issuing the command: **source("filename")** at the R prompt.

Illustrative Examples

Example-1.Let us write codes in R to determine area and perimeter of a Circular plot.

Task Analysis.

We know the solution of this easy problem. The perimeter is obtained by the formula:$\mathbf{2\pi r}$, where,π denotes a known mathematical constant and r denotes the radius of the circular plot.

Implementation of the sequence logic structure in R–Contd.

The area is obtained by the formula: πr^2. So, the only input required is the value of the radius of the circular plot. The value of π is a system-defined constant to be made available by writing **pi**.

So, the instructions required at the R prompt will be as shown below:

>**cat("Enter a radius")**

On execution, this shows: **Enter a radius** >**r=scan(n=1)**

This command is meant for accepting one numeric input for storing in the variable or R object r. On execution, it shows: **1:** and waits for input from the user. If the user types in some value ,say, **5** and presses the **ENTER** key, the screen will appear as shown below:

1: 5

Read 1 item

The following two statements are assignment statements;so R will just accept the commands and perform the desired task internally i.e. calculating the **area** and **perimeter** with the given value of the radius and the constant value of **pi** and then storing the calculated results in **p** and **area** respectively.

Illustrative Examples on Sequence Logic

>**p=2*pi*r**

>**area=pi*r*r**

Now, the command to display the area will be as shown below:

>**cat("The required area is", area)**

On execution of the command, it shows:

The required area is 78.53982 for the given input

For the next required output,we enter the following command:

cat("The Perimeter is:",p,"\n")

On execution of the command, it shows:

The Perimeter is: 31.41593

Let us now execute the following command:

>**print("THANK YOU\n")**

This returns the following text:

[1] "THANK YOU"

Let us now write down all these instructions in a new file as described above and save this file with the name: **circle_area**.The file will be saved automatically with the extension:**.R**,thus making the filename as **circle_area.R**.

Execution process of an R program in batch mode

Now, to run this batch of instructions,we need to enter the command: **source("path_name\filename")** at the command prompt of R.
To know the **path_name**, we should know the current working directory.To get this,we issue the command: **getwd()**.This may show some path like the following: **"[1] C:/Users/ANIL BIKASH/Documents"**.
This implies that the R program file that we have created should be under the folder:**Documents** in the path, unless directed otherwise.
In R, this path has to written as **"C:\\Users\\ANIL BIKASH\\Documents"**
Hence, our command for the execution of the program in batch mode will be:
source("C:\\Users\\ANIL BIKASH\\Documents\\circle_area.R")
This will start the execution of the statements in the program one by one to generate the desired result.However, if the file is not found in the specified path,we need to discover the path by looking at the **properties** of the file location where it is saved.
We can also change the **path** by changing the current working directory with the command: **setwd("New_Path")**

Illustrative Examples on Sequence Logic–Contd.

Example-2. Write a program to determine the volume of a rectangular box.
Task Analysis. The volume of a rectangular box is determined by the product of length, breadth and height. So, we need to accept three inputs from the console The solution is shown below:

```
cat("Enter Length, Breadth and Height")
x=scan(n=3)
v=x[1]*x[2]*x[3]
cat("The required volume is:",v,"cubic units")
```

The program above can also be run at the R console in three steps as illustrated below:

```
>cat("Enter Length, Breadth and Height")
Enter Length, Breadth and Height>x=scan(n=3)
1: 10
2: 20
3: 30
Read 3 items
>v=x[1]*x[2]*x[3]
>cat("The required volume is:",v,"cubic units")
[1] "The required volume is: 6000 cubic units"
```

Illustrative Examples on Sequence Logic–Contd.

Example-3. Develop a program to obtain the area of a triangle.

Task Analysis. The area of any triangle can be obtained in two ways: firstly, if the lengths of all sides are known; secondly, if the lengths of the base and the altitude are known.

In the first method, we first determine: s=$\frac{a+b+c}{2}$, i.e. half of the perimeter, if a, b, c are the lengths of three sides and then obtain the area from the formula: area= $\sqrt{s(s-a)(s-b)(s-c)}$.

In the second method, the formula is: Area=$\frac{base*altitude}{2}$. As the first method is a bit complex, we shall demonstrate its use. The coding of the method is shown below:

```
cat("Enter Length, Breadth and Height of the triangle")
l=scan(n=3)
s=(l[1]+l[2]+l[3])/2
area=sqrt(s*(s-l[1])*(s-l[2])*(s-l[3]))
cat("The required area is:",area, "square units")
```

The program is run in two selections, first the **cat()** and **scan()** statements are selected for execution and then the remaining statements, it will show

Illustrative Examples on Sequence Logic–Contd.

the following in the console:

cat("Enter Length, Breadth and Height of the triangle")

Enter Length, Breadth and Height of the triangle>l=scan(n=3)

1: 2

2: 9

3: 10

Read 3 items

>s=(l[1]+l[2]+l[3])/2

>area=(s*(s-l[1])*(s-l[2])*(s-l[3]))

>cat("The required area is:",area,"square units")

[1] "The required area is: 66.9375 square units"

>s=(l[1]+l[2]+l[3])/2

>print(s)

[1] 10.5

>area=sqrt(s*(s-l[1])*(s-l[2])*(s-l[3]))

>print(paste("The required area is:",area,"square units"))

[1] "The required area is: 8.18153408597679 square unit

Illustrative Examples on Sequence Logic–Contd.

It may be noted that we have used a library function **sqrt()** in the coding. The other statements are easily understandable as explained in other contexts.

Example-4. If P amount of money is invested for N years at a rate of interest I, compounded annually, the money grows to an amount T. It is required to develop a program in R to determine the value of T.

Task Analysis. Taking P, N, I and T as described in the problem, we know that the compounded value can be obtained by the formula: $T = P(1+\frac{I}{100})^N$.

The following R code shows the solution of the problem:

cat(" Enter the Principal amount P, Annual Rate of Interest I and the term N in Years")

Enter the Principal amount P, Annual Rate of Interest I and the term N in Years>x=scan(n=3)

1: 20000

2: 7.5

3: 5

Read 3 items

Continuation of the preceding example on sequence logic

```
>T=x[1]*(1+x[2]/100)^x[3]
>cat(" The compounded value of the amount",x[1]," At annual rate of interest",x[2]," after",x[3],"years,compounded annually will be: Rs.",T)

[1] " The compounded value of the amount 20000 At annual rate of interest 7.5 after 5 years, compounded annually will be: Rs. 28712.5865234375"
>print("Thank You")
[1] "Thank You"
```

Example-5. It is required to develop a program to determine the volume and surface area of a sphere.

Task Analysis. We know that the volume of a sphere is given by V= $(\frac{4}{3})\pi R^3$, where, π is the usual mathematical constant, R is the radius of the sphere. The surface area of the sphere can be obtained by the formula: SA=$4\pi R^2$, where π and R have the usual meaning. So, the determination of the volume and the surface area can be done easily if the radius of the sphere is known.

Continuation of the preceding example on sequence logic

The following code shows the solution:

```
cat("Enter the radius of the sphere")
R=scan(n=1)
V=(4/3)*pi*R^3
SA=4*pi*R^2
print(paste("The required volume of the sphere with radius ",R,"is: ",V,"cubic units"))
print(paste("The surface area of the sphere is: ",SA,"square units"))
```

Instead of **cat()**, we can also use **print(paste())** as shown above.

```
print("Thank You")
```

EXAMPLE-6. It is required to develop R script to interchange the values stored in any two R variables.

Task analysis. In R environment, any two values, be they numbers or string of characters, can be interchanged easily with the help of a third variable as explained below.

Continuation of the preceding example on sequence logic

The task comprises storing the value of the first variable (it may be the second variable also) in any newly defined third variable and then storing the value of the other variable i.e. that of the second variable (or that of the first variable, if the value in the third variable is that of the first one) into the first variable(or into the second variable in case the option within the bracket is followed), then the value of the third variable can be copied into the second variable(into the first variable for alternative action). Thus the third variable is used to hold the value to be transferred temporarily.
In case of numeric values, we have an alternative. The task can also be performed without using an intermediate variable following the procedure shown below assuming that the variables are : a and b :

a=a+b

b=a-b

a=a-b

The following code shows the required R script for the solutions:

Continuation of the preceding example on sequence logic

```
# R script to interchange any two values in various methods
cat("Enter any two numbers:")
x=scan(n=2)
a=x[1]
b=x[2]
t=x[1]
x[1]=x[2]
x[2]=t
cat(" The Given values are:",a,"and",b)
cat("The interchanged values are:",x[1],"and",x[2])
# Now we shall interchange two character strings
x=readline(prompt="Enter the first string:=>")
y=readline(prompt="Enter the second string:=>")
i=x
x=y
y=i
print(paste("The interchanged strings are: ",x,"and",y))
print("End of program...Thank You")
```

Illustrative Examples on Sequence Logic–Contd.

The output displayed on the R console when the above is run in steps remembering that the input statement is to be executed through the first selection and the rest of the code through the second selection. Also, if there exists two or more consecutive input statements, then the execution is to be done one statement at a time.

Example-7. The employer of a firm assures the employees of a yearly payment of an amount P after the retirement until he/she dies. Assuming a fixed rate of inflation of r percent per year and the expectancy e of his/her life, The set of steps required to calculate the present value of the total payments from the employer, is to determined.

Task Analysis: This is a problem of determining the present value of an annuity where an annuity is a fixed sum of money paid to someone each year, typically for the rest of his/ her life.

There is a present value of annuity formula to determine the value of a series of future periodic payments at a given time. The present value of annuity formula is based on the concept of time value of money, in that one rupee of present day is worth more than that same rupee at a future date.

Continuation of the preceding example

The formula is:PV=P$\frac{(1-(1+r)^{-n}}{r}$, where, P=periodic payments, r= Percent rate of inflation or discounting per period/100, n=number of periods. Here, we shall require the age of retirement and the expectancy of life to determine the value of n for which the annuity will be received by a person. The other input requirements will be amount of annuity P and the rate of discount of the money known as rate of inflation. We can then evaluate the expression to determine the present value. The following figure shows the R script required to solve the problem in general.

```
# R script for the determination of Present Value of an annuity
cat("Enter the Age of retirement followed by life expectancy")
a=scan(n=2)
cat("Enter the amount of annuity followed by the rate of inflation")
b=scan(n=2)
n=a[2]-a[1]
P=b[1]
r=b[2]/100
PV=P*((1-(1+r)^(-n))/r)
cat(" The present value of an yearly annuity of Rs.",P,"is Rs.",PV,"for a
period of ",n,"years at a constant inflation rate of ", r)
print("End of program.....Thank You")
```

Illustrative Examples on Sequence Logic–Contd.

Example-8. A program in R is required to be developed to calculate the **EMI** of a loan amount.

Task Analysis: The term EMI stands for Equated Monthly Installment. It is the amount payable every month to the bank or any other financial institution until the loan amount is fully paid off. It consists of the interest on loan as well as part of the principal amount to be repaid. The sum of principal amount and interest is divided by the tenure, i.e., number of months, in which the loan has to be repaid. This amount has to be paid monthly. The interest component of the EMI would be larger during the initial months and gradually reduce with each payment. The exact percentage allocated towards the payment of the principal depends on the interest rate. Even though the monthly EMI payment won't change, the proportion of principal and interest components will change with time. With each successive payment, the borrower pays more towards the principal and less in interest.

Here's the formula to calculate EMI:E$=P.r.\frac{(1+r)^n}{(1+r)^(n)-1)}$ Where,

E is EMI, and **P is Principal Loan Amount**

r is rate of interest calculated on monthly basis. (i.e., r = Rate of Annual interest/12/100. If rate of interest is 10.5% per annum, then r = 10.5/12/100=0.00875)

n is loan term / tenure / duration in number of months.

Continuation of the preceding example

The required R script for the purpose of EMI calculation on a given loan amount has been coded below:

Program to calculate the EMI on a loan amount using the formula:E=$P.r.\frac{(}{(1+}$

cat("Enter the Loan amount followed by yearly percent rate of interest followed by term in years")

v=scan(n=3)

P=v[1]

r=v[2]/1200 # This is to determine the monthly rate of interest over each rupee

n=v[3]*12 # This is to convert the term in years into the equivalent term in months

E=P*r*((1+r)^n)/((1+r)^n -1)

cat("The required equated monthly installment will be Rs. ",E)

print("End of program.....Thank you")

Example-9. An R script is required to be developed for evaluating the yearly amount of depreciation on any asset value.

Continuation of the preceding example

Task Analysis: Depreciation is a reduction in the value of an asset over time, due to its wear and tear in course of its use over time. Straight line depreciation is the default method used to gradually reduce the carrying amount of a fixed asset over its useful life. The method is designed to reflect the consumption pattern of the underlying asset over time. Under the straight-line method of depreciation, we distribute the depreciation expense evenly over the estimated useful life of an asset. The straight-line calculation steps are as stated below:

1. The initial cost of the asset that has been recognized as a fixed asset is determined first. It may be buying price of the asset. It is called the **cost of the asset.**
2. A value called the **salvage value of the asset** is then estimated which denotes the value at which the asset can be sold out at the end of its useful life. In other words, Salvage value (also called residual value or **scrap value**) is the estimated value of the fixed asset at the end of its useful life.
3. The **useful life** of the asset is then estimated.
4. we use the following formula to determine the value of depreciation for each year: D= $\frac{Cost-salvage-value}{Useful-life-in-years}$

Continuation of the preceding example

The following R script is required for the calculation of depreciation in Straight Line Method:

```
# Program to calculate depreciation using the Straight Line Method.
cat("Enter the cost of the asset followed by scrap value followed by Life in years")
v=scan(n=3)
d=(v[1]-v[2])/v[3]
cat(" The Depreciation amount using SLM is Rs.: ",d)
print("Thank you")
```

Example-10. The relationship between the three standard scales of temperate, namely, Centigrade scale(C), Fahrenheit scale (F), and Reumer Scale is given by: $\frac{C}{5} = \frac{F-32}{9} = \frac{R}{4}$

It is required to develop a program in R to convert available temperatures in the Celsius scale (C) into its equivalent temperature in the other two scales.

Task Analysis: Here the task is very easy and straightforward. We take the relationship between the Centigrade (also known as Celsius scale) and the Fahrenheit scale.

Continuation of the preceding example

From the equation, we derive the following relation easily:
F=$\frac{9C}{5} + 32$
Similarly, by taking the relation between the Fahrenheit and Reumer scale, we can obtain:
R=$\frac{4(F-32)}{9}$
The following R codes have been developed for the purpose:
Program for conversion of temperature values from Fahrenheit scale to the others

```
cat("Enter the temperature in Celcius Scale")
t=scan(n=1)
f=9*t/5 + 32
r=4*(f-32)/9
cat(" The equivalent Fahrenheit temperature is: ",f,"and the equivalent Reumer value is:",r)
print("End of Program and Thank You")
```

Outcome of this Lesson

The **coveted outcome** of this lesson are the following:

- **The features of a computer program**
- **The concepts of Assembler,Compiler,Linker,Loader, Source module,Object module and executable module**
- The concept of an **interpreted programming environment**
- The **features of structured and unstructured programs**
- **The basic concepts of Top-Down programming approach and Moduler programming**
- **The program logic structures used in a structured program** and
- Some illustrative examples with the implementation of the **'sequence logic'** structure